

# ClapCap: Lightweight Prefix-Conditioned Language Models for Music Captioning via a CLAP-to-LM Architecture

Ali Ozkaya

Department of Computer Engineering  
Izmir Institute of Technology

## Abstract

We study whether a *lightweight* language model (LM) can describe music when fed nothing but a short, learned prefix projected from a frozen audio embedding. Our pipeline, CLAPCAP, is the audio analogue of ClipCap [1]: a frozen CLAP audio encoder produces a single clip embedding, a small projection network maps it to a fixed-length prefix of pseudo-tokens, and a pretrained LM autoregressively generates the caption. On MusicCaps [2] we run a controlled comparison of two LM families — GPT-2 (decoder-only, 124M) and T5-base (encoder-decoder, 220M) — holding the encoder, projection, data split and evaluation fixed so that the LM is the only variable. We report a ten-metric suite with FENSE [3] as the primary score, and we measure a per-model *noise floor* from three seeds so that every ablation delta can be judged against run-to-run variance. We find that (i) the two LMs are statistically indistinguishable at baseline once the noise floor is accounted for; (ii) second-stage fine-tuning erases nearly all architectural differences introduced in the projection stage; (iii) the largest decoding lever, sentence completion, improves FENSE mostly by removing the fluency-error term rather than by improving semantics; and (iv) the models capture genre, instrumentation and mood well but share a systematic male-vocalist bias, which we trace to caption-prior amplification under uncertain audio evidence rather than to a missing audio cue. We frame the work as an efficiency- and analysis-oriented study of the minimum viable architecture for music captioning, not a state-of-the-art claim. Code is available at [github.com/raccoon-42/audio-caption](https://github.com/raccoon-42/audio-caption).

## 1. Introduction

Most audio captioning research targets *general* sound — environmental events and acoustic scenes — on AudioCaps [4] and Clotho [5]. Music is different: a useful music caption describes *musical* attributes such as genre, instrumentation, mood, tempo and structure rather than discrete sound events. This paper asks a deliberately minimal question: *can a lightweight language model understand music semantics through a simple projection from a frozen audio embedding?*

Our approach, CLAPCAP, adapts ClipCap [1], which captions images by projecting a frozen CLIP embedding into a prefix of pseudo-tokens for GPT-2. We replace the

image encoder with a frozen CLAP audio encoder [6], caption music instead of images, and — crucially — turn the single-LM recipe into a *controlled comparison* of language models. Because the encoder, projection, data split and evaluation are all shared, the language model is the only thing that changes between conditions.

The contributions of this paper are:

- An audio adaptation of the ClipCap prefix-captioning recipe to music, with a single shared evaluation harness.
- A *noise-floor* methodology: per-model run-to-run variance from three seeds, so single-run ablation deltas are interpretable rather than anecdotal.
- A controlled comparison of a decoder-only (GPT-2) and an encoder-decoder (T5) LM, with ablations over projection architecture and decoding strategy.
- A qualitative and statistical analysis of a recurring vocalist bias, which we trace to caption-prior amplification under uncertain audio evidence rather than to the audio frontend alone.

This paper does not claim state-of-the-art performance. Published captioning systems are evaluated on different datasets with different reference counts, so a numerical comparison would be meaningless (Section 2), and we do not attempt one. The contribution is the controlled analysis itself, together with the efficiency of the recipe — only a small projection (plus the LM) is trained.

## 2. Related Work

**Prefix captioning.** ClipCap [1] maps a frozen CLIP image embedding to a constant-length prefix ( $K=10$  tokens) consumed by GPT-2, training only a small mapping network. It offers two design points: a lightweight transformer mapper with a frozen LM, and an MLP mapper with a fine-tuned LM. CLAPCAP is the audio analogue of the MLP-mapper, fine-tuned-LM variant. Notably, ClipCap reports that this variant *overfits* as trainable capacity grows — an observation we recover, amplified, when scaling the LM (Section 6). In the audio domain, Pengi [7] likewise conditions a small LM on an audio-derived prefix, framing a range of audio tasks as text generation.

**General audio captioning.** Audio captioning is typically benchmarked on AudioCaps [4] and Clotho [5],

which describe general sounds and provide multiple references per clip. These differ from music captioning in domain, number of references, and the metrics commonly reported (CIDEr/SPIDEr rather than the semantic FENSE). Cross-dataset numerical comparison is therefore not meaningful, and we avoid it.

**Music captioning.** MusicCaps [2] is a hand-curated set of 5,521 ten-second clips from AudioSet [8], each annotated by musicians with a multi-sentence caption and a list of musical aspects. LP-MusicCaps [9] addresses caption scarcity by generating pseudo-captions with an LLM over tag datasets and trains a convolutional-encoder/BART-decoder captioner. Such domain-specific captioners are a natural *reference* for CLAPCAP, but a fair comparison must run them on our exact test split and avoid the leakage that arises when a captioner is itself trained on MusicCaps; we leave this to future work.

**Audio language models.** End-to-end audio LLMs such as Qwen-Audio [10] and SALMONN [11] couple a large LM to an audio encoder and are trained on large instruction corpora. They are far heavier than the present recipe; we regard them as an upper-bound reference direction rather than a controlled comparison, since they bypass the frozen-encoder/learned-prefix design we study.

### 3. Method

**Overview.** A clip is encoded once by a frozen CLAP model into a  $d_a$ -dimensional embedding. A projection network  $f_\theta$  maps this vector to a prefix  $p_1, \dots, p_k \in \mathbb{R}^{d_{lm}}$  of  $k$  pseudo-tokens in the LM embedding space. The LM then generates the caption autoregressively, conditioned on the prefix (Figure 1).

**Audio encoder.** We use the LAION CLAP checkpoint `laion/clap-htsat-unfused` [6], whose audio tower is an HTS-AT transformer [12] producing a 512-dimensional audio embedding. CLAP is trained on general audio and is *not* music-specialised — a deliberate test of how far a general frontend carries on a music task, and a limitation we revisit. Embeddings are precomputed once and cached, so the encoder never runs during training or evaluation. The “unfused” variant ingests a fixed  $\sim 10$ s window, matching MusicCaps clip length.

**Projection network.** The projection is a small MLP,  $\text{Linear} \rightarrow \text{LayerNorm} \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}$ , reshaped to  $(k, d_{lm})$ . The baseline uses prefix length  $k=8$ , dropout 0.3 and depth 2.

**Language models.** We compare GPT-2 [13] (decoder-only, 124M) and T5-base [14] (encoder-decoder, 220M). For T5 the prefix is fed to the encoder; for GPT-2 it is prepended as input embeddings. Extending the same harness to modern billion-parameter decoders is in progress (Section 6).

**Two-stage training.** In Stage 1 (S1) the LM is frozen and only the projection is trained; in Stage 2 (S2) the projection and the full LM are fine-tuned together (the audio encoder stays frozen). S1 corresponds to ClipCap’s frozen-LM regime and S2 to its fine-tuned-LM regime. The objective in both stages is autoregressive cross-entropy on cap-

tion tokens conditioned on the prefix,

$$\mathcal{L} = - \sum_j \log p_\theta(c_j \mid p_1, \dots, p_k, c_1, \dots, c_{j-1}).$$

We use AdamW [15] (weight decay 0.1), cosine annealing [16], batch size 8 and early stopping (patience 5). Per model, only the learning rates are tuned, via a 16-trial Optuna [17] search over an 8-epoch proxy; all else is fixed so the LM stays the controlled variable.

**Efficiency.** The only newly added module is the projection ( $\sim 10.2$ M parameters): Stage 1 trains it alone with the LM frozen, and Stage 2 additionally fine-tunes the LM. The complete model is 124M (GPT-2) or 220M (T5) parameters — one to two orders of magnitude smaller than the billion-parameter end-to-end audio LLMs — and every experiment in this paper runs on a single consumer GPU (NVIDIA RTX 5090). Because CLAP embeddings are precomputed once, the audio encoder is never run during training.

## 4. Experimental Setup

**Data.** We use MusicCaps [2] with a fixed three-way random split: a held-out test set ( $n=529$ , final metrics only), a validation set ( $n \approx 495$ , for checkpoint selection and hyperparameter search), and the remainder for training ( $n \approx 4,455$ ). The split seed is fixed across all conditions, so every model and seed is scored on the identical test set.

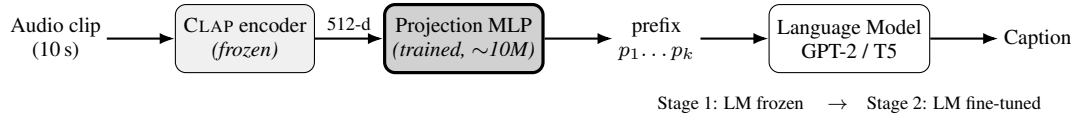
**Metrics.** A single shared harness scores all conditions with the `aac-metrics` suite: BLEU-1 and BLEU-4 [18], METEOR [19], ROUGE-L [20], CIDEr-D [21], SPICE [22], SPIDEr [23], SBERT-sim, FER and FENSE. FENSE [3] is our primary metric: it combines a SentenceBERT [24] semantic similarity with a fluency-error penalty (FER), making it sensitive to the semantic correctness a listener cares about. We apply no post-processing by default; sentence trimming is treated as an explicit decoding ablation (Section 5.4).

**Noise floor.** To judge whether a delta is real, we retrain each baseline with three seeds (42, 43, 44), varying only initialisation, shuffling and dropout while holding the data split fixed. The standard deviation of each metric across seeds is our *noise floor*; deltas smaller than it are not interpreted as effects.

## 5. Results

### 5.1 Baseline comparison

Table 1 reports greedy-decoding test metrics with per-model noise floors. GPT-2 leads on nine of the ten metrics, including the primary FENSE (0.3998 vs. 0.3954); the exception is FER, where T5 is lower (better), though that gap (0.023) sits inside T5’s own FER noise floor ( $\pm 0.0511$ ). The FENSE margin (0.004) is likewise smaller than either model’s FENSE noise floor ( $\pm 0.0137$  for GPT-2,  $\pm 0.0345$  for T5). We therefore do *not* read it as evidence that one architecture is better at music captioning under this recipe. The noise floor is also  $\sim 2.5\times$  larger for T5 than GPT-2, so



**Figure 1.** The CLAPCAP pipeline. A frozen CLAP encoder turns the audio clip into a single 512-d embedding; a small projection MLP (the only newly added module) maps it to a  $k$ -token prefix in the LM embedding space; the LM generates the caption. Stage 1 trains the projection with the LM frozen; Stage 2 additionally fine-tunes the LM.

the encoder–decoder is noticeably noisier run-to-run and each model must be judged against its *own* variance.

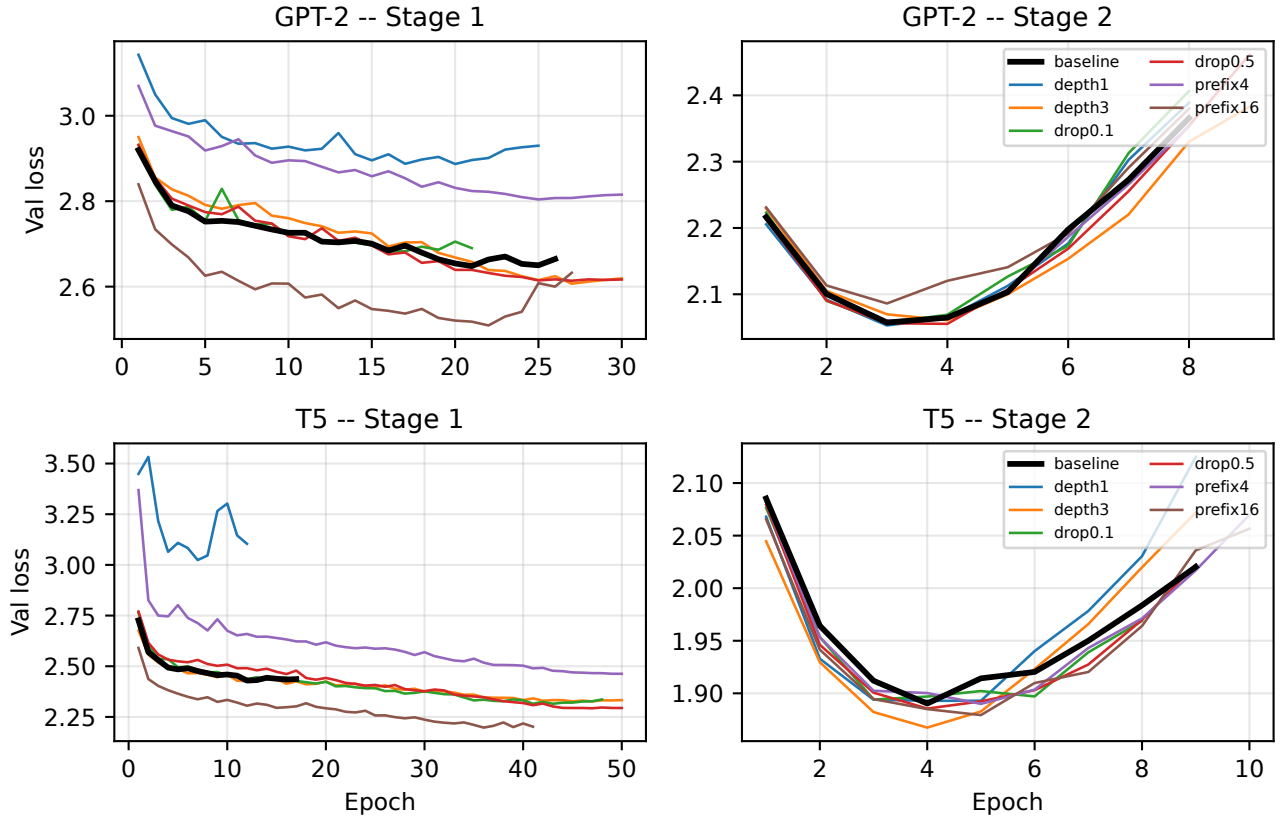
**Table 1.** Baseline (greedy) test-set metrics for both language models, with per-model run-to-run standard deviation (noise floor) from three seeds. FENSE is the primary metric; best per column in bold.  $\uparrow$ : higher is better;  $\downarrow$ : lower is better.

| Model | BLEU-1 $\uparrow$ | BLEU-4 $\uparrow$ | METEOR $\uparrow$ | ROUGE-L $\uparrow$ | CIDEr-D $\uparrow$ | SPICE $\uparrow$ | SPIDEr $\uparrow$ | SBERT-sim $\uparrow$ | FER $\downarrow$ | FENSE $\uparrow$                      |
|-------|-------------------|-------------------|-------------------|--------------------|--------------------|------------------|-------------------|----------------------|------------------|---------------------------------------|
| GPT-2 | <b>0.3091</b>     | <b>0.0631</b>     | <b>0.1265</b>     | <b>0.2329</b>      | <b>0.1109</b>      | <b>0.1091</b>    | <b>0.1100</b>     | <b>0.5820</b>        | 0.3478           | <b>0.3998 <math>\pm 0.0137</math></b> |
| T5    | 0.3003            | 0.0574            | 0.1162            | 0.2275             | 0.0847             | 0.0866           | 0.0857            | 0.5564               | <b>0.3251</b>    | 0.3954 $\pm 0.0345$                   |

*The GPT-2/T5 FENSE gap (0.004) is smaller than either model’s noise floor, i.e. not significant.*

## 5.2 Stage 1 vs. Stage 2

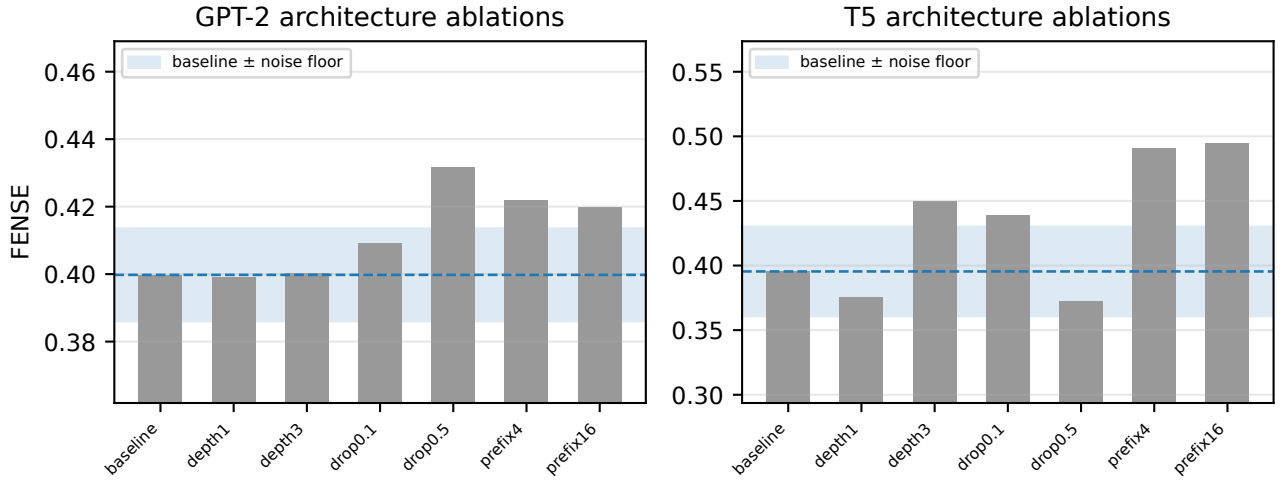
Figure 2 shows validation loss per epoch for every projection configuration. In Stage 1, where only the projection is trained, the configurations spread widely (a range of 0.38 for GPT-2 and 0.83 for T5). After Stage 2 fine-tuning that spread collapses to 0.03 for both models: full fine-tuning largely erases the architectural differences introduced by the projection. The S2 curves also reveal early overfitting — validation loss bottoms within a few epochs and then rises — which early stopping handles.



**Figure 2.** Validation loss per epoch for all projection configurations. *Left:* Stage 1 (projection only) shows a wide spread across configurations. *Right:* Stage 2 (full fine-tuning) collapses that spread — the configurations become nearly indistinguishable — and shows the early-overfitting onset. The baseline is drawn in bold.

### 5.3 Architecture ablations

We vary prefix length ( $\{4, 16\}$  around 8), dropout ( $\{0.1, 0.5\}$  around 0.3) and projection depth ( $\{1, 3\}$  around 2), retraining each from scratch with greedy decoding. Figure 3 plots the resulting FENSE against the baseline  $\pm$  noise-floor band. Consistent with Section 5.2, most configurations fall within the band: no projection hyperparameter reliably moves the primary metric for GPT-2. For T5 the prefix variants produce larger FENSE swings, but — as the next section shows — these coincide with collapses in the fluency-error term rather than genuine semantic gains.

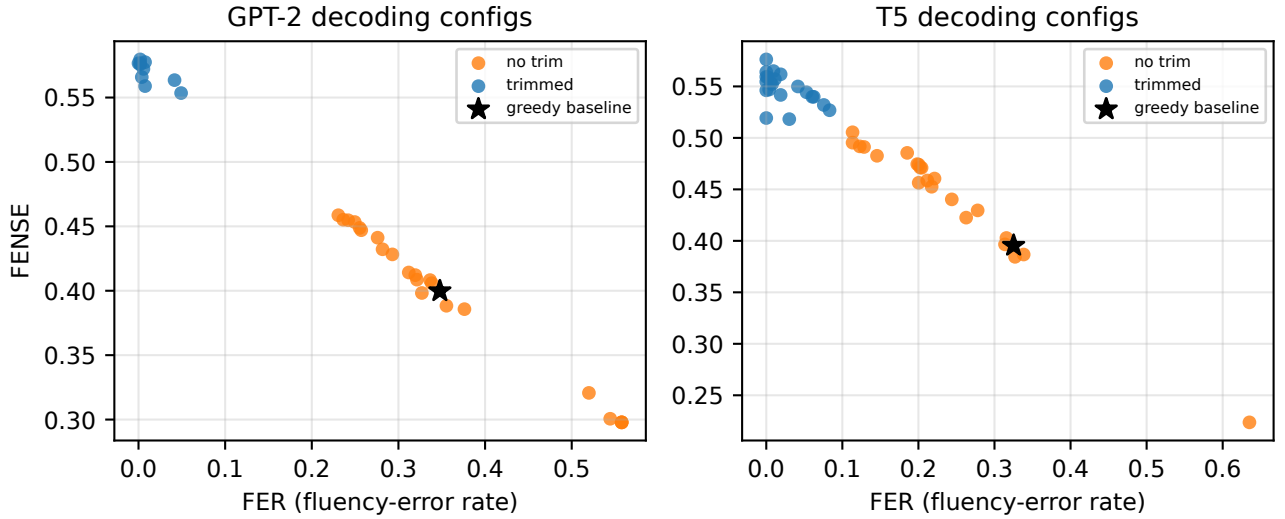


**Figure 3.** Architecture-ablation FENSE per configuration. The shaded band is the baseline  $\pm$  one noise-floor standard deviation. Bars inside the band are not distinguishable from the baseline.

#### 5.4 Decoding ablations

Decoding strategies are evaluated on the fixed baseline checkpoint (no retraining), isolating generation behaviour. Table 2 lists the top configurations by FENSE. The dominant lever is trimming trailing incomplete sentences: it lifts FENSE from 0.3998 to 0.5796 for GPT-2 and from 0.3954 to 0.5762 for T5.

This gain is, however, *largely an artifact of the metric*. Figure 4 plots FENSE against FER across all decoding configurations: the relationship is almost perfectly inverse, and the trimmed configurations simply drive FER to near zero while the lexical metrics in Table 2 barely move. In other words, trimming removes the fluency penalty inside FENSE without making captions semantically more correct. We report it as a useful decoding choice for producing clean captions, but caution against reading the headline FENSE jump as a semantic improvement.



**Figure 4.** FENSE vs. FER over all decoding configurations. The near-perfect inverse trend shows that FENSE here is dominated by the fluency-error term: trimmed configurations (low FER) score high FENSE without improving semantic similarity. The star is greedy decoding.

**Table 2.** Top decoding configurations per model (eval-only on the baseline checkpoint), ranked by FENSE. Note that high FENSE coincides with near-zero FER while the lexical metrics barely move – see Figure 4.  $\uparrow$ : higher is better;  $\downarrow$ : lower is better.

| Config                   | BLEU-1 $\uparrow$ | BLEU-4 $\uparrow$ | METEOR $\uparrow$ | ROUGE-L $\uparrow$ | CIDEr-D $\uparrow$ | SPICE $\uparrow$ | SPIDEr $\uparrow$ | SBERT-sim $\uparrow$ | FER $\downarrow$ | FENSE $\uparrow$ |
|--------------------------|-------------------|-------------------|-------------------|--------------------|--------------------|------------------|-------------------|----------------------|------------------|------------------|
| <b>GPT-2</b>             |                   |                   |                   |                    |                    |                  |                   |                      |                  |                  |
| greedy (baseline)        | 0.3091            | 0.0631            | 0.1265            | 0.2329             | 0.1109             | 0.1091           | 0.1100            | 0.5820               | 0.3478           | 0.3998           |
| trim_ngram2              | 0.3171            | 0.0575            | 0.1210            | 0.2205             | 0.1092             | 0.1081           | 0.1087            | 0.5807               | 0.0019           | <b>0.5796</b>    |
| trim_rep1.2_ngram2_t0.3  | 0.2829            | 0.0454            | 0.1137            | 0.1970             | 0.1003             | 0.1062           | 0.1033            | 0.5812               | 0.0076           | 0.5775           |
| trim_rep1.2_ngram2       | 0.2810            | 0.0409            | 0.1124            | 0.1931             | 0.0959             | 0.1042           | 0.1000            | 0.5766               | 0.0000           | 0.5766           |
| trim_rep1.2_ngram3       | 0.2839            | 0.0407            | 0.1127            | 0.1944             | 0.0926             | 0.1045           | 0.0986            | 0.5781               | 0.0019           | 0.5766           |
| trim_rep1.2              | 0.2834            | 0.0408            | 0.1124            | 0.1945             | 0.0924             | 0.1043           | 0.0983            | 0.5774               | 0.0019           | 0.5758           |
| trim_rep1.3              | 0.2605            | 0.0374            | 0.1058            | 0.1843             | 0.0885             | 0.0990           | 0.0938            | 0.5748               | 0.0057           | 0.5723           |
| <b>T5</b>                |                   |                   |                   |                    |                    |                  |                   |                      |                  |                  |
| greedy (baseline)        | 0.3003            | 0.0574            | 0.1162            | 0.2275             | 0.0847             | 0.0866           | 0.0857            | 0.5564               | 0.3251           | 0.3954           |
| trim_rep1.2              | 0.3045            | 0.0498            | 0.1158            | 0.2203             | 0.0863             | 0.0921           | 0.0892            | 0.5762               | 0.0000           | <b>0.5762</b>    |
| trim_beam5_ngram2        | 0.1923            | 0.0428            | 0.0918            | 0.1967             | 0.0878             | 0.1013           | 0.0945            | 0.5698               | 0.0095           | 0.5649           |
| trim_rep1.3              | 0.2934            | 0.0424            | 0.1124            | 0.2065             | 0.0779             | 0.0987           | 0.0883            | 0.5640               | 0.0000           | 0.5640           |
| trim_rep1.2_beam5_ngram2 | 0.1956            | 0.0442            | 0.0941            | 0.1949             | 0.0985             | 0.1023           | 0.1004            | 0.5716               | 0.0189           | 0.5618           |
| trim_rep1.2_ngram2       | 0.2527            | 0.0296            | 0.0982            | 0.1904             | 0.0662             | 0.1000           | 0.0831            | 0.5593               | 0.0000           | 0.5593           |
| trim_rep1.2_ngram2_t0.3  | 0.2500            | 0.0331            | 0.0978            | 0.1897             | 0.0611             | 0.0973           | 0.0792            | 0.5599               | 0.0019           | 0.5591           |

## 5.5 Qualitative analysis

Table 3 shows representative successes and failures. The models reliably recover high-level musical content — genre (rock, jazz, electronic), instrumentation (electric guitar, bass, drums, piano) and mood (groovy, energetic, emotional). Two failure modes recur: on *non-music* audio the pipeline still emits a confident music caption, and greedy decoding occasionally degenerates into verbatim repetition. Most strikingly, both models tend to insert “a male vocalist singing melodically” even when the reference explicitly states there is no singer.

To characterise this we measured vocal-presence and vocalist-gender agreement over the full test set. The bias is *not* a blind spot: GPT-2 mentions vocals on 95% of clips whose reference describes singing, but also on 43% of clips marked instrumental (T5: 82%/29%) — both models track vocal presence well above chance while skewing

toward false positives. The skew mirrors the training captions, of which 53% mention vocals and in which male vocalists outnumber female ones 2.2:1; the phrase “male vocalist” appears in 10% of training captions but in roughly half of the generated ones. Gender is worse: on references describing a *female* singer, T5 outputs “male” 89% of the time and GPT-2 44%, while male references are almost never misgendered (2–4%). These rates persist across decoding configurations, so they are not a decoding artifact.

To separate the frontend from the LM, we probed the frozen CLAP embeddings directly with zero-shot text prompts: vocal vs. instrumental is separable at 74% accuracy and vocalist gender at 76% (female recall only 55%). The embedding therefore carries the relevant cues, but noisily — and since both LMs read the *same* embedding, the gap between GPT-2 (56%) and T5 (11%) in female-vocalist accuracy shows the failure is largely model-side. We conclude that the vocalist bias is caption-prior amplifi-

cation under uncertain audio evidence, not a missing audio cue: a music-specialised encoder should sharpen the evidence, but the male-vocalist default itself is learned from the caption statistics.

**Table 3.** Representative successes and failures (baseline greedy; captions trimmed at the last full sentence for display). The models recover genre, instrumentation and mood, but default to a male vocalist even on instrumental or non-music audio.

| Successes        |                                                                                                                                                                                                                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Reference</i> | instrumental rock music with a drum kit, lots of cymbal crashes, acoustic guitar strumming, harmonized lead guitars and an active bass guitar.                                                                                                                                                                   |
| <i>GPT-2</i>     | This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing the main tune while the bass guitar is playing in the background. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy.                                            |
| <i>T5</i>        | This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing a simple tune with a bass guitar playing in the background. The rhythmic background consists of a fast-paced acoustic drum beat.                                                                        |
| <i>Reference</i> | This is a drum & bass electronic music piece. There is no singer. There is a strong and fuzzy feeling bass sound. The rhythmic background consists of a loud and high tempo electronic drum beat. The atmosphere is modern and energetic. This piece could be playing at a nightclub or a dance club.            |
| <i>GPT-2</i>     | This is a techno music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass. The rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is groovy. This piece could be used in the background of nightclubs.                             |
| <i>T5</i>        | This is a techno house music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass while the rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is energetic.                                                                         |
| <i>Reference</i> | A female vocalist sings this melancholic melody in a foreign language. The tempo is slow with a romantic piano accompaniment. The song is soft, mellow, poignant, emotional, sentimental, romantic, melancholic, sad, lonely, and wistful. This song is a Pop/ Love song.                                        |
| <i>GPT-2</i>     | This is a lullaby piece. There is a female vocalist singing melodically. The piano is playing a simple tune while the bass guitar is playing in the background. The atmosphere is calming and relaxing. This piece could be used in the background of a meditation video.                                        |
| <i>T5</i>        | This is a classical music piece. There is a female vocalist singing melodically. The melody is being played by the piano while the rhythm is being played by the acoustic guitar. The atmosphere is romantic.                                                                                                    |
| <i>Reference</i> | The low quality recording features a live performance of a jazz song and it consists of a groovy bass guitar, groovy rhodes keys, electric guitar melody, shimmering hi hats, punchy kick and punchy snare hits. It sounds groovy, funky and passionate. The recording is noisy and in mono.                     |
| <i>GPT-2</i>     | This is a jazz music piece. There is a saxophone solo being played. The melody is being played by the piano and the electric guitar. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy. This piece could be used in the background of a jazz bar.                        |
| <i>T5</i>        | This is a live recording of a jazz music piece. There is a saxophone playing the lead melody while a bass guitar is playing in the background. The rhythm is being played by the acoustic drums. The atmosphere is jazzy.                                                                                        |
| Failure cases    |                                                                                                                                                                                                                                                                                                                  |
| <i>Reference</i> | The low quality recording features a printer that produces printing sounds and some muffled music playing in the background, probably on a radio. It is noisy and in mono.                                                                                                                                       |
| <i>GPT-2</i>     | This is a recording of a live performance of a pop music piece. There is a male vocalist singing melodically. The melody is being played by a keyboard and a bass guitar while the rhythmic background is provided by a fast tempo electronic drum beat.                                                         |
| <i>T5</i>        | This is a live recording of a DJ performance. The DJ is scratching the turntable repeatedly. The turntable is a low quality recording. The sound of the turntable is distorted and the recording is of very poor quality.                                                                                        |
| <i>Reference</i> | The low quality recording features a metallic hit, followed by resonating bowl tone. The recording is mono and noisy.                                                                                                                                                                                            |
| <i>GPT-2</i>     | This is a meditation music piece. There is a bowl-like sound being played by the bowl. The bowl is being rubbed repeatedly. There is a calming and relaxing atmosphere to this piece. This piece can be used in the background of meditation videos. It can also be used in the background of meditation videos. |
| <i>T5</i>        | This is a Tibetan singing bowl playing in the background of a meditation video. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played.                                                                      |

## 6. Limitations and Future Work

**General-purpose audio frontend.** Our encoder is trained on general audio, not music. A music-specialised encoder (a music CLAP checkpoint, MERT [25] or MusicFM [26]) is a natural and likely beneficial swap, especially for the fine-grained attributes (vocalist gender, instrument identity) where the embedding’s cues are noisiest (Section 5.5).

**Capacity vs. data.** Scaling the LM is not free in this low-data regime ( $\sim 4.5k$  clips). In preliminary runs a billion-parameter decoder fine-tuned end-to-end overfits within a single epoch — the same MLP-plus-fine-tuning overfitting that ClipCap [1] reports, amplified by scale — and requires a gentler schedule to remain comparable. This supports the lightweight framing and motivates our in-progress extension to such models.

**Metric caveats.** MusicCaps provides a single reference per clip, which mechanically depresses n-gram metrics relative to multi-reference benchmarks. And as Section 5.4 shows, FENSE can be inflated by fluency-driven trimming; we mitigate this by reporting FER and lexical

metrics alongside it.

**Reference and human evaluation.** We do not yet compare against a domain-specific music captioner on our split; the clean route is a leakage-free checkpoint evaluated on our exact test items via the same harness. We also plan an LLM-as-judge and human evaluation of semantic correctness, reporting inter-rater agreement.

## 7. Conclusion

We presented CLAPCAP, a controlled study of lightweight language models for *music* captioning via an audio adaptation of the ClipCap prefix recipe. Holding the frozen CLAP encoder, the projection, the data split and the evaluation fixed, we found that a decoder-only and an encoder-decoder LM are statistically indistinguishable at baseline once run-to-run variance is measured; that second-stage fine-tuning erases projection-level differences; and that the largest decoding lever improves the primary metric mostly by removing a fluency penalty rather than by improv-

ing semantics. A reproducible vocalist bias turns out to be caption-prior amplification under uncertain audio evidence; sharpening that evidence with a music-specialised frontend is the natural next step. The results characterise the minimum viable architecture for music captioning and the care needed to interpret its metrics, rather than chasing state of the art.

## 8. References

- [1] R. Mokady, A. Hertz, and A. H. Bermano, “ClipCap: CLIP Prefix for Image Captioning,” *arXiv:2111.09734*, 2021. 1, 9
- [2] A. Agostinelli, T. I. Denk, Z. Borsos, *et al.*, “MusicLM: Generating Music From Text,” *arXiv:2301.11325*, 2023. 1, 2
- [3] Z. Zhou, Z. Zhang, X. Xu, *et al.*, “Can Audio Captions Be Evaluated with Image Caption Metrics?,” in *Proc. ICASSP*, 2022. 1, 2
- [4] C. D. Kim, B. Kim, H. Lee, and G. Kim, “AudioCaps: Generating Captions for Audios in The Wild,” in *Proc. NAACL-HLT*, 2019. 1
- [5] K. Drossos, S. Lipping, and T. Virtanen, “Clotho: An Audio Captioning Dataset,” in *Proc. ICASSP*, 2020. 1
- [6] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, “Large-Scale Contrastive Language-Audio Pre-training with Feature Fusion and Keyword-to-Caption Augmentation,” in *Proc. ICASSP*, 2023. 1, 2
- [7] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, “Pengi: An Audio Language Model for Audio Tasks,” in *Proc. NeurIPS*, 2023. 1
- [8] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An Ontology and Human-Labeled Dataset for Audio Events,” in *Proc. ICASSP*, 2017. 2
- [9] S. Doh, K. Choi, J. Lee, and J. Nam, “LP-MusicCaps: LLM-Based Pseudo Music Captioning,” in *Proc. ISMIR*, 2023. 2
- [10] Y. Chu, J. Xu, X. Zhou, *et al.*, “Qwen-Audio: Advancing Universal Audio Understanding via Unified Large-Scale Audio-Language Models,” *arXiv:2311.07919*, 2023. 2
- [11] C. Tang, W. Yu, G. Sun, *et al.*, “SALMONN: Towards Generic Hearing Abilities for Large Language Models,” in *Proc. ICLR*, 2024. 2
- [12] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, and S. Dubnov, “HTS-AT: A Hierarchical Token-Semantic Audio Transformer for Sound Classification and Detection,” in *Proc. ICASSP*, 2022. 2
- [13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI, Tech. Rep., 2019. 2
- [14] C. Raffel, N. Shazeer, A. Roberts, *et al.*, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *JMLR*, vol. 21, no. 140, 2020. 2
- [15] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. ICLR*, 2019. 2
- [16] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *Proc. ICLR*, 2017. 2
- [17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-Generation Hyperparameter Optimization Framework,” in *Proc. ACM SIGKDD*, 2019. 2
- [18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a Method for Automatic Evaluation of Machine Translation,” in *Proc. ACL*, 2002. 2
- [19] S. Banerjee and A. Lavie, “METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments,” in *Proc. ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 2005. 2
- [20] C.-Y. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004. 2
- [21] R. Vedantam, C. L. Zitnick, and D. Parikh, “CIDEr: Consensus-Based Image Description Evaluation,” in *Proc. CVPR*, 2015. 2
- [22] P. Anderson, B. Fernando, M. Johnson, and S. Gould, “SPICE: Semantic Propositional Image Caption Evaluation,” in *Proc. ECCV*, 2016. 2
- [23] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, “Improved Image Captioning via Policy Gradient Optimization of SPIDER,” in *Proc. ICCV*, 2017. 2
- [24] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019. 2
- [25] Y. Li, R. Yuan, G. Zhang, *et al.*, “MERT: Acoustic Music Understanding Model with Large-Scale Self-Supervised Training,” in *Proc. ICLR*, 2024. 9
- [26] M. Won, Y.-N. Hung, and D. Le, “A Foundation Model for Music Informatics,” in *Proc. ICASSP*, 2024. 9